GUIA VISUAL 009

NAVEGAÇÃO E CAMINHOS

O comando do colega funciona na máquina dele e falha na sua. Não mudou o comando — mudou de onde ele foi executado.

009
GUIAS LINUX

MESMO DESTINO, DOIS CAMINHOS

```
/
├─ etc
│  └─ ssh
│     └─ sshd_config
├─ home
│  └─ kali
│     └─ projetos
│        └─ recon   ← pwd
└─ var
   └─ log
```

ABSOLUTO — COMEÇA EM `/`

`/etc/ssh/sshd_config`

Aponta para o mesmo arquivo **de qualquer lugar**. É o que se usa em script e em cron.

RELATIVO — PARTE DO `PWD`

`../../../etc/ssh/sshd_config`

Digita menos, mas **só vale a partir de `recon`**. Um `cd` antes e o mesmo texto aponta para outro lugar — ou para lugar nenhum.

O diretório atual é um campo do *processo*, não do usuário nem do terminal. Dá para vê-lo por fora em `/proc/<PID>/cwd` — útil para achar quem segura um sistema de arquivos que você quer desmontar. Daí saem três consequências: cada aba tem o seu; **o filho herda e nunca devolve** — um script que faz `cd` move a si mesmo e morre, o seu shell fica onde estava (por isso `source`); e é por isso que `cd` precisa ser *builtin*. Já `.` e `..` são **entradas reais** do sistema de arquivos, não sintaxe — é o que faz `./script.sh` rodar sem o diretório atual estar no `PATH`.

A LINHA DO `LS -L`, CAMPO A CAMPO

`-`	`rw-r--r--`	`1`	`root`	`root`	`2891`	`Jul 14 09:22`	`/etc/passwd`
TIPO - arquivo · d dir · 1 link	PERMISSÕES dono · grupo · outros	LINKS quantos nomes apontam ao inode	DONO	GRUPO	TAMANHO bytes (-h converte)	MODIFICAÇÃO do conteúdo	NOME

OS ATALHOS E OS COMANDOS QUE VIRAM REFLEXO

SÍMBOLO	SIGNIFICA
`.`	O diretório atual
`..`	O diretório pai
`~`	Seu diretório pessoal — **não expande dentro de aspas**; em script use `"$HOME"`
`~kali`	O diretório pessoal do usuário `kali`
`-`	O diretório anterior (guardado em `OLDPWD`), só com `cd`
`cd`	Sem argumento nenhum: vai para o seu `~`

`cd -`
Alterna entre os dois últimos diretórios. O atalho que mais rende no dia a dia.

`pushd dir · popd · dirs -v`
Pilha de diretórios, para quando são mais de dois pontos na investigação.

`ls -lah`
O padrão de trabalho: longo, com ocultos, tamanho legível.

`ls -ltr`
Mais recente **por último**, coladinho no prompt. O reflexo de quem lê log.

`ls -ld /var/log`
A linha **do próprio diretório**, não o conteúdo — é o que responde "por que não consigo entrar aqui".

`pwd -P · readlink -f · realpath`
Resolvem links e `..` até o caminho físico. `pwd` sozinho mostra o caminho **lógico**, por onde você chegou — e depois de entrar por um link, `cd ..` vai para o pai do *link*, não do alvo.

RESUMO RÁPIDO

- ✓ Absoluto começa em `/` e vale sempre; relativo depende do `pwd`.
- ✓ Script que faz `cd` não move o seu shell — para mover, `source`.
- ✓ Oculto é só nome com ponto: sem `-a` o seu `~` parece vazio.
- ✓ `-F` marca o tipo (`/` dir, `*` exec, `@` link) e **sobrevive ao cano**, quando a cor não.
- ✓ Script que lê arquivo ao lado dele: `DIR="$(cd -- "$(dirname -- "$0")" && pwd -P)"`.

NO KALI

O `~` do usuário é `/home/kali`; o do root é `/root` — depois de `sudo -i` o que você salvar **não está no seu diretório**. É a explicação de "salvei o relatório e não acho mais". O Zsh do Kali liga o `AUTO_CD`: digitar só `/var/log` já muda de diretório (não funciona em Bash nem em script), e `cd /u/l/b` + Tab expande para `/usr/local/bin`. Ferramenta clonada do GitHub vai para `/opt`, que **não está no** `PATH` — é o caso de uso nº 1 do `pushd` / `popd`. Wordlists em `/usr/share/wordlists` (a `rockyou.txt` vem `.gz` numa instalação nova). E faça de `ls -la ~/.ssh` rotina: chave com permissão frouxa faz o cliente SSH **recusar** a conexão.